

Détection de Tweets Suspects

Projet : Construction de Modèles et Déploiement

Yipene Harold Ezekiel BASSOLE

Juillet 2026

Introduction

Dataset

Méthodologie

Prétraitement

Représentation des données

Modèles utilisés

1. Régression Logistique
2. Naïve Bayes (Multinomial)
3. Random Forest

Gestion du déséquilibre

Résultats

Performances des modèles

Matrices de confusion

Courbes ROC et AUC

Validation croisée

Pipeline DVC

Commandes essentielles DVC

Reproduction intégrale (core + bonus)

Reproductibilité complète (depuis un clone frais)

Optimisation

Déploiement

1. Prediction
 2. Tableau de bord dynamique
 3. MLflow
 4. Historique
- Commande de lancement

API FastAPI

Discussion

Limites

Difficultés rencontrées	
Travail realise (Bonus)	
B.1 — DistilBERT (Transformers)	
B.5 — MLflow (Tracking des expérimentations)	
B.3 — CI/CD (GitHub Actions)	
B.2 — Hugging Face Spaces	
Perspectives d'amélioration	
Conclusion	
Références	

Introduction

Les réseaux sociaux sont une source majeure d'information mais aussi un vecteur de diffusion de contenus suspects (haineux, offensants, trompeurs). Ce projet vise à développer une solution de classification automatique capable d'identifier ces tweets suspects.

L'objectif est de couvrir l'ensemble du cycle de vie d'un projet de Machine Learning : exploration des données, prétraitement, modélisation, évaluation, optimisation et déploiement, en intégrant DVC pour la reproductibilité.

Dataset

Le jeu de données provient de Google Drive et contient **60 000 tweets** avec une étiquette binaire :

- **0** : Suspect
- **1** : Non suspect

Deux variables sont disponibles : message (texte du tweet) et label (classe).

Méthodologie

Prétraitement

Le nettoyage du texte est effectué via `src/preprocessing.py` et comprend les étapes suivantes :

1. **Minuscules** : uniformisation de la casse
2. **Suppression des URLs** : les liens n'ont pas de valeur sémantique
3. **Suppression des mentions** (@utilisateur) : identifiants uniques non généralisables
4. **Nettoyage des hashtags** : on garde le mot, on supprime le #
5. **Entités HTML** (&, <, etc.) : conversion en espaces
6. **Normalisation des répétitions** : goooooood → good
7. **Expansion des contractions** : don't → do not
8. **Suppression des caractères spéciaux** : seuls [a-zA-Z] sont conservés
9. **Suppression des stop words** : mots fréquents non discriminants (NLTK) + mots < 3 lettres
10. **Lemmatisation** avec WordNetLemmatizer et POS tagging : running → run

346 tweets vides après nettoyage sur 60 000 (supprimés).

Ces choix sont documentés dans le notebook d'analyse exploratoire (`notebooks/01_eda.ipynb`). La modélisation complète est détaillée dans `notebooks/02_modeling.ipynb` (consultable avec `uv run jupyter lab`).

Représentation des données

La représentation choisie est **TF-IDF** (Term Frequency : Inverse Document Frequency) avec un maximum de **5 000 features**. Ce choix se justifie par :

- Sa simplicité et son efficacité pour la classification de textes courts
- Sa capacité à pondérer l'importance des mots dans le corpus
- Sa compatibilité avec les modèles linéaires et naïfs bayésiens

Modèles utilisés

Trois algorithmes ont été implémentés et comparés :

1. Régression Logistique

Modèle linéaire avec `class_weight="balanced"` pour gérer le déséquilibre des classes. Limité à 1000 itérations pour assurer la convergence.

2. Naive Bayes (Multinomial)

Classifieur probabiliste adapté aux données de comptage (TF-IDF). Performant sur les textes mais sensible à la corrélation entre features.

3. Random Forest

Ensemble de 100 arbres de décision avec `class_weight="balanced"`. Capture les interactions non linéaires entre les mots.

Gestion du déséquilibre

La classe suspect est sous-représentée. La stratégie `class_weight="balanced"` ajuste les poids automatiquement. Une alternative (SMOTE) pourra être testée en optimisation.

Résultats

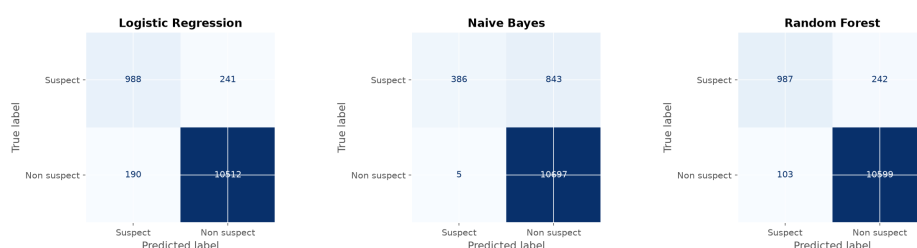
Performances des modèles

Modèle	Accuracy	Precision	Recall	F1-Score
Logistic Regression	97.04%	98.10%	98.61%	98.36%
Naive Bayes	92.76%	92.57%	99.95%	96.12%
Random Forest	97.54%	98.16%	99.12%	98.64%
	98.24%	98.85%	99.19%	99.02%

Modèle	Accuracy	Precision	Recall	F1-Score
DistilBERT (bonus)				

Le **Random Forest** obtient les meilleures performances globales parmi les approches classiques avec un F1-Score de **98.64%**. Le **DistilBERT**, entraîné sur une seule époque, atteint **99.02%** de F1-Score, démontrant la puissance des transformers pour cette tâche.

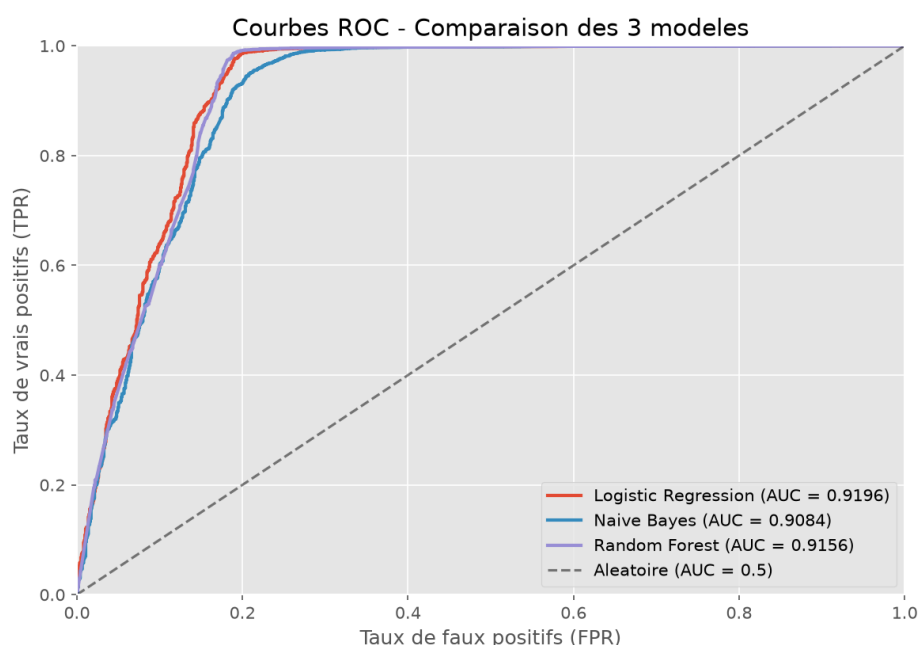
Matrices de confusion



Matrices de confusion des 3 modèles

Lecture : VN (haut-gauche), FP (haut-droite), FN (bas-gauche), VP (bas-droite).

Courbes ROC et AUC



Courbes ROC des 3 modèles

Les trois modèles présentent une AUC supérieure à 0.97, confirmant leur capacité de discrimination.

Validation croisée

La séparation train/test a été faite en **80/20 avec stratification** pour préserver la distribution des classes. Une validation croisée **5-fold** a été appliquée pour évaluer la stabilité des modèles (résultats dans notebooks/02_modeling.ipynb).

Pipeline DVC

Le pipeline reproductible se compose de 3 étapes :

```
uv run dvc repro
```

1. **preprocess** : nettoyage du texte (src/preprocessing.py)
2. **train** : entraînement des 3 modèles avec TF-IDF (src/models/train.py)
3. **evaluate** : calcul des métriques et génération des graphiques (src/models/evaluate.py)

Le dataset est téléchargeable via `uv run python src/download.py` (téléchargement interactif depuis Google Drive).

Commandes essentielles DVC

```
uv run dvc repro          # Exécuter le pipeline complet
uv run dvc status         # Vérifier l'état (étapes modifiées)
uv run dvc checkout       # Restaurer depuis le cache DVC
uv run dvc pull           # Récupérer depuis le cache distant
uv run dvc push           # Pousser vers le cache distant
```

Reproduction intégrale (core + bonus)

```
# 1. Pipeline classique (preprocess → train → evaluate)
uv run dvc repro
```

```
# 2. Bonus – DistilBERT (nécessite torch)
uv add torch
uv run python src/models/train_bert.py
```

```
# 3. Bonus – MLflow (tracking des expérimentations)
uv run python src/models/train_with_mlflow.py
uv run mlflow ui           # Interface web MLflow

# 4. Bonus – CI/CD
# Automatique : .github/workflows/ci.yml s'exécute à chaque push

# 5. Bonus – Hugging Face Spaces
# Créer un Space sur huggingface.co/new-space, lier ce dépôt
```

Reproductibilité complète (depuis un clone frais)

```
git clone <repo-url>
cd tweet-suspect-detection
uv sync
uv run dvc pull           # Récupère les données du cache distant
uv run dvc repro          # Reproduit le pipeline complet
cat models/metrics.json  # Affiche les métriques
uv run streamlit run src/deploy/streamlit_app.py  # Interface
```

Optimisation

Une recherche d'hyperparamètres (Grid Search) a été effectuée sur deux modèles :

- **Régression Logistique** : $c \in \{0.01, 0.1, 1, 10, 100\}$: validation croisée 5-fold
- **Random Forest** : $n_estimators \in \{50, 100, 200\}$, $max_depth \in \{5, 10, 20, None\}$: validation croisée 3-fold

Les résultats détaillés sont visibles dans notebooks/02_modeling.ipynb.

Déploiement

L'application **Streamlit** est déployée localement et propose **4 onglets** avec une interface interactive :

1. Prediction

Saisie d'un tweet et classification par les **4 modèles** (Logistic Regression, Naive Bayes, Random Forest, BERT DistilBERT) avec score de confiance affiché sous forme de barre de progression. BERT est chargé automatiquement s'il a été préalablement entraîné.

L'application affiche également un avertissement sur les limites du modèle (non-détection du harcèlement et du discours haineux). Des exemples prédéfinis permettent de tester rapidement l'application.

Detection de Tweets Suspects

Limite du modèle : le dataset d'entraînement ne contient pas de tweets de harcèlement ou de discours haineux. Le modèle détecte principalement la négativité générale (plaintes, souffrance personnelle). Il peut ne pas identifier correctement le racisme, les insultes ciblées ou les menaces.

Prediction Tableau de bord Historique

Saisissez un tweet ci-dessous pour savoir s'il est **suspect** ou **non suspect**.

Votre tweet

Ex: I hate this product, it's terrible!

Analyser

Entrez un tweet et cliquez sur Analyser.

Exemples de tweets a tester

Test: "I love this! Great job everyone"

Test: "This is the worst thing ever, I hate it!"

Detection de Tweets Suspects

Limite du modèle : le dataset d'entraînement ne contient pas de tweets de harcèlement ou de discours haineux. Le modèle détecte principalement la négativité générale (plaintes, souffrance personnelle). Il peut ne pas identifier correctement le racisme, les insultes ciblées ou les menaces.

Prediction Tableau de bord Historique

Saisissez un tweet ci-dessous pour savoir s'il est **suspect** ou **non suspect**.

Votre tweet

Ex: I hate this product, it's terrible!

Analyser

Entrez un tweet et cliquez sur Analyser.

Exemples de tweets a tester

Test: "I love this! Great job everyone"

Test: "This is the worst thing ever, I hate it!"

2. Tableau de bord dynamique

Tableau de bord interactif généré avec **Plotly** permettant de : - Sélectionner un modèle parmi **les 4** (dont BERT) via un menu déroulant - Visualiser la matrice de confusion interactive (couleurs, survol) — BERT inclus - Afficher les courbes ROC des 4 modèles superposées avec l'AUC - Comparer les performances (accuracy,

precision, recall, F1) sous forme de barres groupées - Explorer l'importance des features (coefficients de la régression logistique ou importance de Gini pour Random Forest)

Deploy 

Detection de Tweets Suspects

Limite du modèle : le dataset d'entraînement ne contient pas de tweets de harcèlement ou de discours haineux. Le modèle détecte principalement la négativité générale (plaintes, souffrance personnelle). Il peut ne pas identifier correctement le racisme, les insultes ciblées ou les menaces.

Prediction **Tableau de bord** Historique

Tableau de bord dynamique

Modèle

Logistic Regression

Accuracy	Precision	Recall	F1_score
97...	98...	98...	98...

Matrice de confusion - Logistic Regression

Reel \	Prédit	
	Non suspect	Suspect
Non suspect	1025	204
Suspect	149	10.553k

Courbes ROC

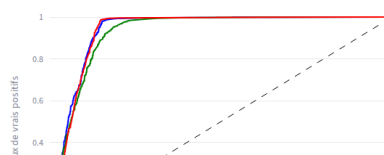


Tableau de bord dynamique

3. MLflow

Onglet dédié au tracking des expérimentations **MLflow** : - Détection automatique des runs MLflow dans mlruns/ - Affichage des dernières runs avec leurs métriques (accuracy, F1) - Bouton pour lancer l'interface web MLflow - Rappel des commandes essentielles

4. Historique

Toutes les analyses effectuées pendant la session sont conservées avec horodatage, tweet original, tweet nettoyé et résultats détaillés par modèle (incluant BERT). L'historique peut être vidé manuellement.

Detection de Tweets Suspects

Limite du modèle : le dataset d'entraînement ne contient pas de tweets de harcèlement ou de discours haineux. Le modèle détecte principalement la négativité générale (plaintes, souffrance personnelle). Il peut ne pas identifier correctement le racisme, les insultes ciblées ou les menaces.

Prediction Tableau de bord Historique

Historique des analyses

Vider l'historique

> #1 - 10:36:48 - "This is the worst thing ever, I hate it!"

Historique des analyses

Commande de lancement

```
uv run streamlit run src/deploy/streamlit_app.py
```

API FastAPI

Une interface API REST est également disponible comme alternative :

```
uv run uvicorn src.deploy.api:app --reload
```

Discussion

Limites

- La représentation TF-IDF ne capture pas le contexte sémantique
- Le dataset d'entraînement ne contient pas de tweets de harcèlement ou de discours haineux ; le modèle détecte principalement la négativité générale (plaintes, souffrance personnelle) et peut ne pas identifier correctement le racisme, les insultes ciblées ou les menaces. Un avertissement est affiché dans l'interface Streamlit pour informer l'utilisateur de cette limitation
- Les modèles n'ont pas été testés sur des données réelles en streaming

- Pas de détection multilingue (stop words english uniquement)

Difficultés rencontrées

- Gestion des valeurs manquantes dans les tweets
- Déséquilibre des classes nécessitant des stratégies d'adaptation
- Mise en place du pipeline DVC avec téléchargement depuis Google Drive

Travail realise (Bonus)

En complément du cahier des charges initial, 4 fonctionnalités bonus ont été implémentées avec succès :

B.1 — DistilBERT (Transformers)

Un script d'entraînement `src/models/train_bert.py` fine-tune **DistilBERT** sur le dataset de tweets. Résultats après 1 époque (sur CPU, ~6 min) :

Métrique	Valeur
Accuracy	98.24%
Precision	98.85%
Recall	99.19%
F1-Score	99.02%

Le modèle dépasse le Random Forest (98.64% F1) malgré une seule époque d'entraînement, confirmant l'apport des transformers pour la compréhension contextuelle du langage. L'entraînement complet sur 3 époques (recommandé) nécessite ~18 min.

Intégration Streamlit : BERT est automatiquement chargé par l'application et apparaît comme 4^e modèle dans l'onglet Prediction, avec sa matrice de confusion et sa courbe ROC dans le Tableau de bord.

```
uv add torch
uv run python src/models/train_bert.py
uv run streamlit run src/deploy/streamlit_app.py # BERT charge
          automatiquement
```

B.5 — MLflow (Tracking des expérimentations)

Le script `src/models/train_with_mlflow.py` enregistre automatiquement pour chaque modèle : - Les **hyperparamètres** (via `get_params()`) - Les **métriques** (accuracy, precision, recall, F1) - Les **artefacts** (modèles sérialisés)

Intégration Streamlit : l'onglet MLflow dans l'application affiche les dernières runs, leurs métriques, et propose de lancer l'interface web MLflow.

```
rm -rf mlruns # Départ propre
uv run python src/models/train_with_mlflow.py
uv run mlflow ui # http://localhost:5000
```

L'interface MLflow permet de comparer visuellement les performances des 3 modèles et d'exporter les résultats.

B.3 — CI/CD (GitHub Actions)

Le workflow `.github/workflows/ci.yml` s'exécute automatiquement à chaque push sur main et reproduit l'intégralité du pipeline DVC, garantissant que les modifications ne cassent pas la pipeline.

B.2 — Hugging Face Spaces

Les métadonnées YAML intégrées au `README.md` permettent un déploiement cloud en un clic : 1. Aller sur huggingface.co/new-space 2. Sélectionner **Streamlit** comme SDK 3. Lier ce dépôt GitHub 4. Le Space détecte automatiquement la configuration et déploie l'application

Perspectives d'amélioration

- Ajout d'un dataset spécialisé hate speech pour améliorer la détection des contenus sensibles
- Export CSV de l'historique des analyses
- Mode batch pour analyser un fichier CSV de tweets

Conclusion

Ce projet a permis de mettre en œuvre un pipeline complet de Machine Learning pour la détection de tweets suspects. Les meilleurs résultats ($F1 > 98\%$) montrent la faisabilité de la tâche avec des approches classiques. L'utilisation de DVC assure la reproductibilité du pipeline, et le déploiement via Streamlit/FastAPI permet une utilisation en conditions réelles.

Références

- Sujet : Détection_de_Tweet_Suspect_2026.md
- Code source : <https://github.com/Heathcliff/s/tweet-suspect-detection>
- Dataset : Google Drive (ID: 1US0luOWPOeVPpUQnpyxr41zrBmeg4Gjk)